environment. In particular, the *ContentProviders* mechanism provides access to the data stored on the device. An application can use data provided by another application using the *ContentProvider* mechanism, or expose its own data using this mechanism.

When a user installs an application, the permissions dialogue will ask the user to agree to grant the permissions needed by the application. It is done once only, so as not to irritate the user. Overall, Android seems to provide a reasonable environment so that running applications from unknown sources is not as risky as I had first feared.

MeeGo—extended access control

I wasn't looking for MeeGo. I was actually interested in KDE Plasma Active (*http://plasma-active.org/*). I like the Plasma netbook environment, and have been looking forward to the Plasma Active environment on tablets. I was surprised to find that a distribution available for testing, provided by *http://plasma-active.basyskom.com/*, is based on MeeGo for the Intel platform. It is still a work in progress, but I was curious about how security issues are handled by MeeGo.

MeeGo also uses the standard Linux environment. However, user-level security is not enough to isolate resources used by an application. The MeeGo access control framework introduced two new types of credentials— a *Resource Token* and *Application Identifier*. The D-Bus interfaces for a real system object are protected, and an application needing a resource needs to have the credentials to access those interfaces.

Access control via D-Bus is used on desktops as well. For example, on modern Linux desktops, the decisions regarding who may update a system, who may use NetworkManager to configure Wi-Fi, etc, are managed using the PolicyKit framework. An application can create new resource tokens and control access to its sensitive resources. The Application Identifier is generated by the package manager, and remains unchanged during the life of the application.

The additional access control mechanisms are implemented using the SMACK (Simplified Mandatory Access Control Kernel) module in the mainline kernel. Any system object, e.g., a file, will be automatically protected with additional authentication by the kernel if it has a SMACK label.

Firefox OS—HTML5

A draft of the Firefox OS' security model is available on *developer.mozilla.org*. The key difference is that it will allow HTML5 applications to integrate with the device's hardware using JavaScript. A side effect is that the need for numerous applications created specifically for a mobile platform, would disappear. The design of Firefox OS assumes that data transfer is slow, expensive, and has a monthly limitation (a common scenario in many countries, including India). Users are likely to keep data services disabled, except when they need to carry out some transaction.

Even though (as Mark Zuckerberg stated) Facebook made a strategic error betting heavily on HTML5 rather than native applications, the time for HTML5 will come. Mozilla hopes to implement the needed API of HTML5 in Firefox OS, so that the experience of Facebook is a thing of the past. Firefox OS may indeed be the ideal platform for India. It would certainly make it easy to select open source applications rather than those that are merely 'free'. A good place to check the current status of HTML5 on your browser is *http://html5test.com/*.

Firefox OS' security model references the documentation of system hardening by Chromium OS, which also is an environment for running Web applications. As in the case of MeeGo, the key concept is the principle of least privilege, and it is implemented using mandatory and role-based access controls. A number of alternatives are available in the kernel, including SMACK, SELinux and TOMOYO. The core functionality needed by Firefox OS is available.

We may conclude that securing a user's or each application's data against accidents or the malicious intent of other applications is a primary design consideration for mobile platforms. Each environment tries to ensure that an application runs effectively in some type of a sandbox. Sharing of data or objects between applications has to be specifically requested and authorised. While security policies are more complex in these environments than on traditional PCs, mobile OSs try to avoid presenting users with unnecessary technical details. As long as a user does not give unnecessary access rights to an application, various mobile platforms provide a pretty safe environment for a user to install and experiment with new applications. END

By: Anil Seth

The author is currently a visiting faculty member at IIT-Ropar, prior to which he was a professor at Padre Conceicao College of Engineering (PCCE) in Goa. He has managed IT and imaging solutions for Phil Corporation (Goa), and worked for Tata Burroughs/TIL. You can find him online at *http://sethanil.com/* and reach him via email at *anil@sethanil.com*.